

CodeGuardian AI

SUMMARY

Build a professional web application called CodeGuardian AI that acts as an intelligent first-level code reviewer. The objective is to catch common problems before the code reaches a human reviewer and explain those problems in a way a developer can understand and fix.

PROJECTS

CodeGuardian AI | AI-powered software engineering quality platform (Portfolio / Python Backend Developer placement project).

- Primary User: Software developers
- Secondary User: Engineering leads / reviewers
- MVP Language Analyzed: Python
- Backend: Django + Django REST Framework
- Database: MySQL
- Frontend: HTML + CSS + Vanilla JavaScript + Bootstrap 5
- AI: Google Gemini API
- Deployment Target: Render
- Repository: GitHub
- Core capabilities include authentication, project management, code submission via paste and .py upload, deterministic static analysis (AST/Ruff/Bandit/Radon), AI-assisted review, quality scoring, review reports, history, and score comparison.

TECHNICAL SKILLS

Python
Django
Django REST Framework
MySQL
HTML5
CSS3
Vanilla JavaScript
Bootstrap 5
Chart.js
Google Gemini API
JWT Authentication
Python AST
Ruff
Bandit
Radon
GitHub
Render
Gunicorn
WhiteNoise
Postman

DOCUMENT VERSION

PROJECT TYPE

AI-powered software engineering quality platform

PRODUCT WORKFLOW

Register → Login → Dashboard → Create Project → Submit Python Code → Validate Code → Static Analysis → Bandit / Ruff / Radon / AST → Normalize Findings → Gemini AI Analysis → Combine Results → Calculate Scores → Save Review → Display Report → User Fixes Code → Submit Again → Compare Results

TECHNOLOGY CONSTRAINTS

Must use: Python, Django, Django REST Framework, MySQL, HTML, CSS, JavaScript, Bootstrap, Chart.js, Google Gemini API; with Python AST, Ruff, Bandit, Radon where appropriate. Do not introduce React, Node.js backend, MongoDB, FastAPI, Flask, Kubernetes, Kafka, RabbitMQ, Elasticsearch, Vector database.